Christmas Lights Digital Circuit

Final Project for the Subject PB170

Masaryk University

Faculty of Informatics

Lejla Džaferbegović
Dáša Pawlasová

7 January 2021

Abstract

Task specification :

Design a circuit that alternates colors by flashing several (more than 3) colored light emitting dioes (LEDs). The change button allows the user to select between several (more than 3) different flashing sequences.

Our design includes 4 different colored LEDs, that is 4 outputs, as well as 8 different flashing sequences. The circuit is controlled by one input button, which enables alternation between different flashing sequences.
Circuit implementation consists of simple physical parts such as logical gates, LEDs, D flip-flops and a push button.

Contents

1 Circuit Design **3**
1.1 Mealy Machine 3
1.2 Christmas Tree Lights based on Mealy Machine 3

2 Circuit Simulation in Logisim **8**

3 Circuit Realization in KiCad **11**

Chapter 1

Circuit Design

1.1 Mealy Machine

Formally, Mealy machine is a 6-tuple (S, S_0, X, Y, T, G) consisting of the following:

- a finite set of states S
- a start state (also called initial state) S_0 which is an element of S
- a finite set called the input alphabet X
- a finite set called the output alphabet Y
- a transition function T: $S \times X \rightarrow S$ mapping pairs of a state and an input symbol to the corresponding next state
- an output function G: $S \times X \rightarrow Y$ mapping pairs of a state and an input symbol to the corresponding output symbol.

1.2 Christmas Tree Lights based on Mealy Machine

The circuit design is based on Mealy machine, where the output values are determined by both, current state and inputs. This circuit has one input

and four outputs. In our design, we used 4 different LEDs (yellow, green, red and blue) and 8 flashing sequences. The first part of the circuit consists of a push button and a 3-bit DFF counter enabling switching between different flashing sequences by one button push. The second part consists of logical AND gates, DFFs and LEDs. Here, we are analyzing the behaviour of the second part of the circuit by coding all required variables. The inputs and states are coded as given in tables 1.1 and 1.2. Outputs are coded as shown in table 1.3. Table 1.4 represents input-state mapping. Table 1.5 represents mapping of a state and input symbol to corresponding next state, as well as the output.

Input	I_1	I_2	I_3
i_1	0	0	0
i_2	0	0	1
i_3	0	1	0
i_4	0	1	1
i_5	1	0	0
i_6	1	0	1
i_7	1	1	0
i_8	1	1	1

Table 1.1: Coding the inputs

State	R_1	R_2
Q_1	0	0
Q_2	0	1
Q_3	1	0
Q_4	1	1

Table 1.2: Coding the states

Output	Y	G	R	B
Y	1	0	0	0
G	0	1	0	0
R	0	0	1	0
B	0	0	0	1
YG	1	1	0	0
RB	0	0	1	1
YR	1	0	1	0
GB	0	1	0	1
ON	1	1	1	1
OFF	0	0	0	0

Table 1.3: Coding the outputs

I_1	I_2	I_3	Q_1	Q_2	Q_3	Q_4
0	0	0	OFF	OFF	OFF	OFF
0	0	1	Y	G	R	B
0	1	0	B	R	G	Y
0	1	1	Y	R	G	B
1	0	0	YG	RB	YG	RB
1	0	1	YR	GB	YR	GB
1	1	0	ON	OFF	ON	OFF
1	1	1	ON	ON	ON	ON

Table 1.4: Input to state mapping

The expressions for states and outputs were computed using Karnaugh maps, which will not be shown in this document. We are giving the final expressions for states as well as outputs.

- $R_1^+ = R_1\bar{R}_2 + \bar{R}_1 R_2$
- $R_2^+ = \bar{R}_1\bar{R}_2 + R_1\bar{R}_2$
- $Y = I_1 I_2 I_3 + I_1\bar{R}_2 + I_3\bar{R}_1\bar{R}_2 + \bar{I}_1 I_2 \bar{I}_3 R_1 R_2$
- $G = I_1 I_2 I_3 + I_2 R_1 \bar{R}_2 + I_1 \bar{I}_3 \bar{R}_2 + \bar{I}_2 I_3 \bar{R}_1 R_2 + I_1 \bar{I}_2 I_3 R_2$
- $R = I_1 \bar{I}_2 I_3 \bar{R}_2 + I_1 \bar{I}_2 \bar{I}_3 R_2 + \bar{I}_2 I_3 R_1 \bar{R}_2 + I_1 I_2 I_3 + I_1 I_2 \bar{R}_2 + \bar{I}_1 I_2 \bar{R}_1 R_2$

- $B = I_1 I_2 I_3 + I_1 I_2 \bar{R}_2 + I_1 \bar{I}_2 R_2 + \bar{I}_2 I_3 R_1 R_2 + I_2 I_3 R_1 R_2 + \bar{I}_1 I_2 \bar{I}_3 \bar{R}_1 \bar{R}_2$

State diagram is very simple, since we have only 4 states and the transition among them is dependent on input.

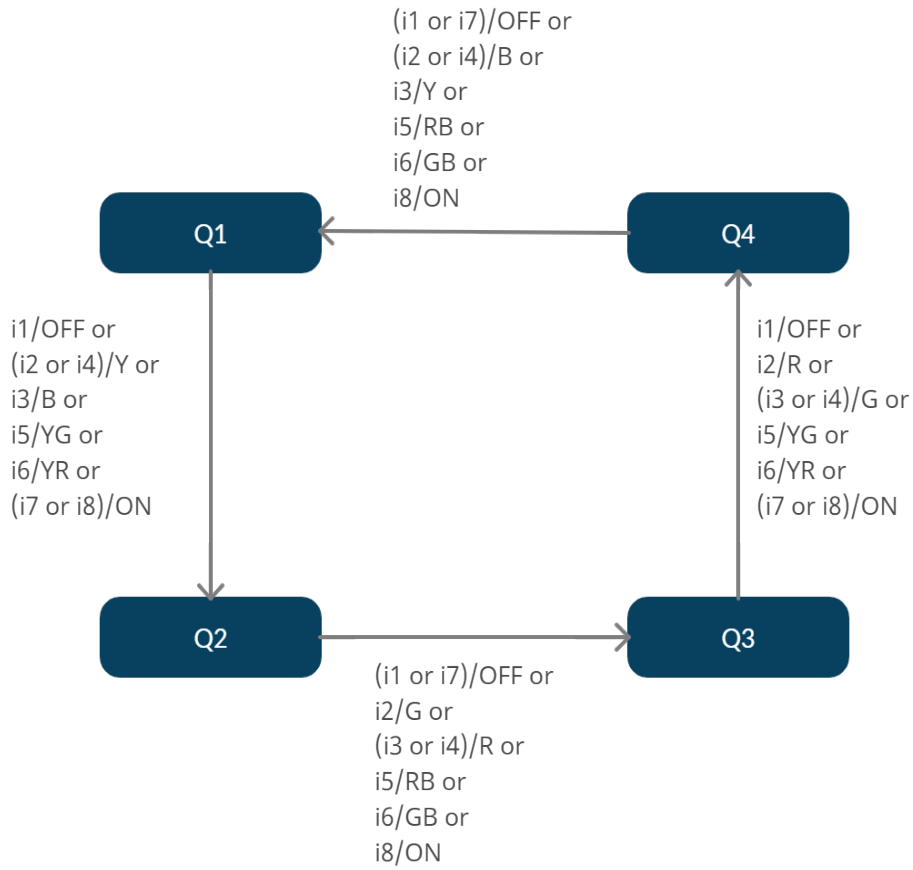

Figure 1.1: State Diagram

R_1	R_2	I_1	I_2	I_3	R_1^+	R_2^+	Y	G	R	B
0	0	0	0	0	0	1	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
1	0	0	0	0	1	1	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	1	1	0	0	0
0	1	0	0	1	1	0	0	1	0	0
1	0	0	0	1	1	1	0	0	1	0
1	1	0	0	1	0	0	0	0	0	1
0	0	0	1	0	0	1	0	0	0	1
0	1	0	1	0	1	0	0	0	1	0
1	0	0	1	0	1	1	0	1	0	0
1	1	0	1	0	0	0	1	0	0	0
0	0	0	1	1	0	1	1	0	0	0
0	1	0	1	1	1	0	0	0	1	0
1	0	0	1	1	1	1	0	1	0	0
1	1	0	1	1	0	0	0	0	0	1
0	0	1	0	0	0	1	1	1	0	0
0	1	1	0	0	0	1	0	0	1	1
1	0	1	0	0	1	1	1	1	0	0
1	1	1	0	0	0	0	0	0	1	1
0	0	1	0	1	0	1	1	0	1	0
0	1	1	0	1	1	0	0	1	0	1
1	0	1	0	1	1	1	1	0	1	0
1	1	1	0	1	0	0	0	1	0	1
0	0	1	1	0	0	1	1	1	1	1
0	1	1	1	0	1	0	0	0	0	0
1	0	1	1	0	1	1	1	1	1	1
1	1	1	1	0	0	0	0	0	0	0
0	0	1	1	1	0	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1
1	0	1	1	1	1	0	1	1	1	1
1	1	1	1	1	0	0	1	1	1	1

Table 1.5: Transition mapping

Chapter 2

Circuit Simulation in Logisim

The circuit schema made in Logisim is shown on Figure 2.1. The sequence for the input 011 is B-Y-R-G as shown on Figures 2.2, 2.3, 2.4 and 2.5.

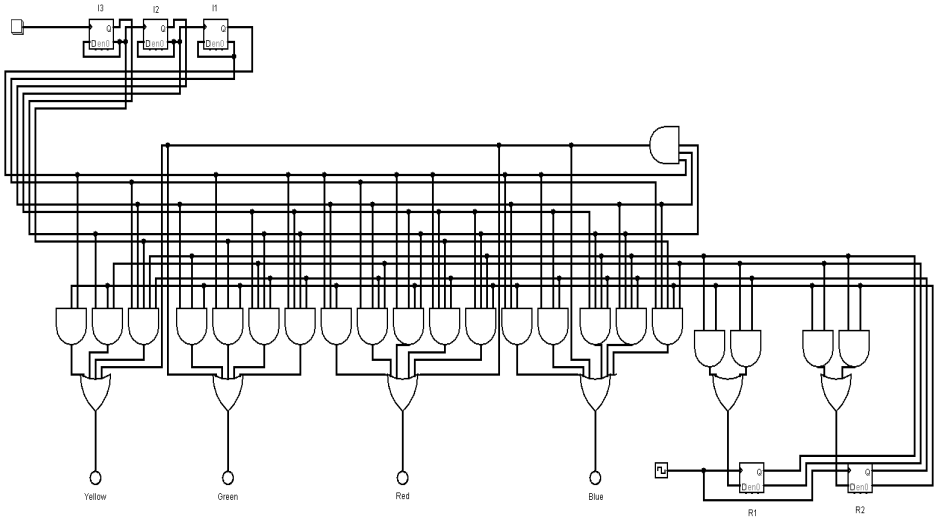

Figure 2.1: Circuit Design in Logisim

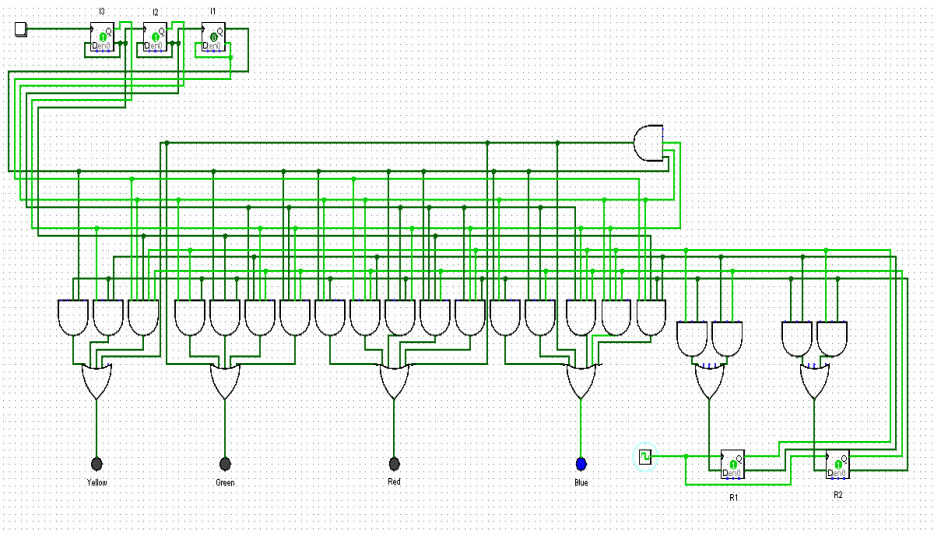

Figure 2.2: Blue LED on

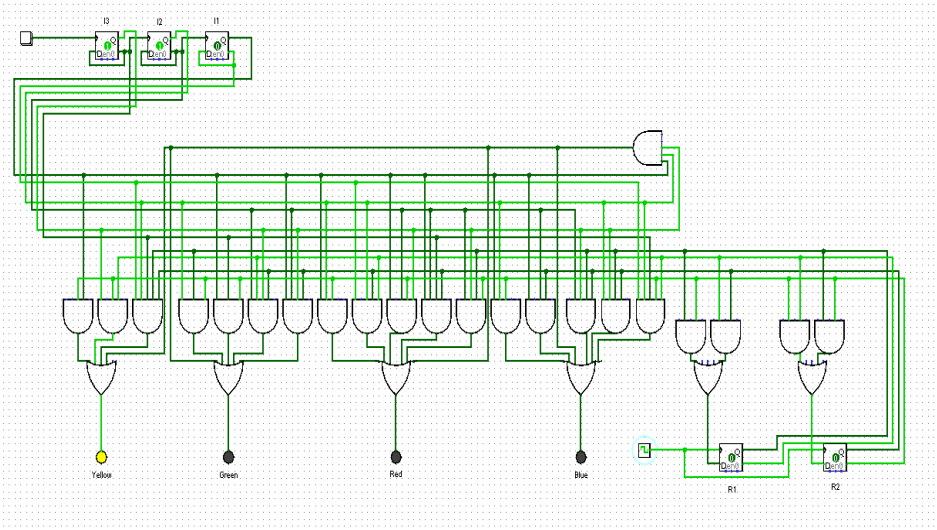

Figure 2.3: Yellow LED on

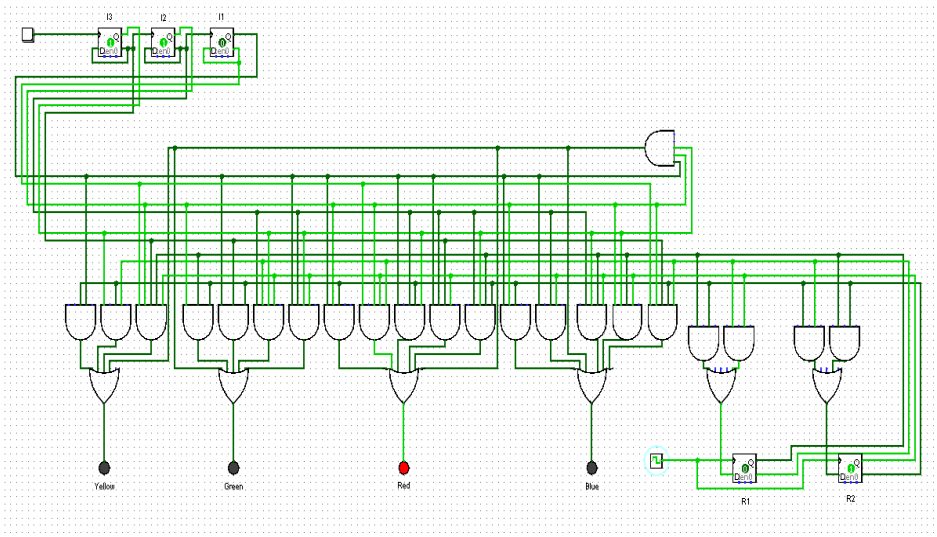

Figure 2.4: Red LED on

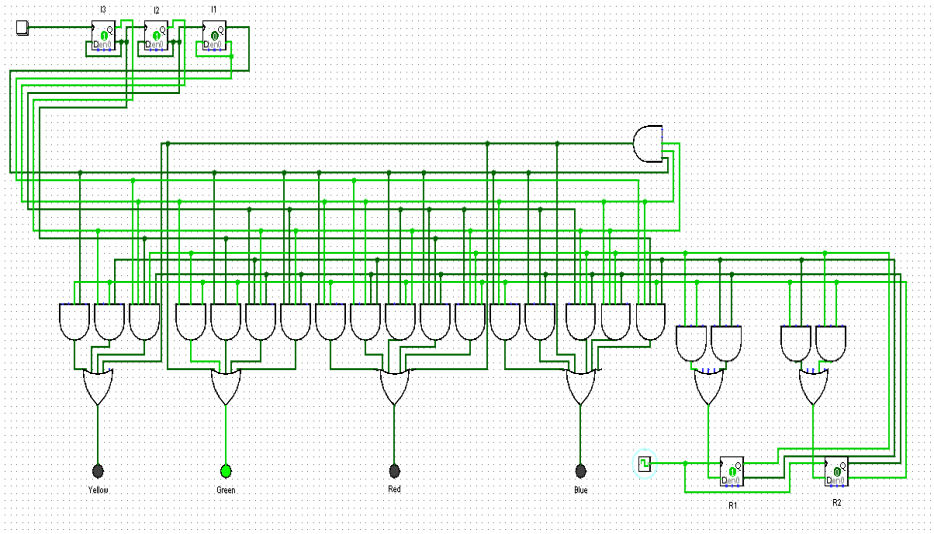

Figure 2.5: Green LED on

Chapter 3

Circuit Realization in KiCad

The circuit design in KiCad was seperated into hierarchical sheets for simplicity. The general scheme is shown on Figure 3.1. Separate sheets are shown on Figures 3.2, 3.3, 3.4, 3.5, 3.6 and 3.7. Corresponding PCB scheme and 3D model of the circuit are shown on Figures 3.8 and 3.9.

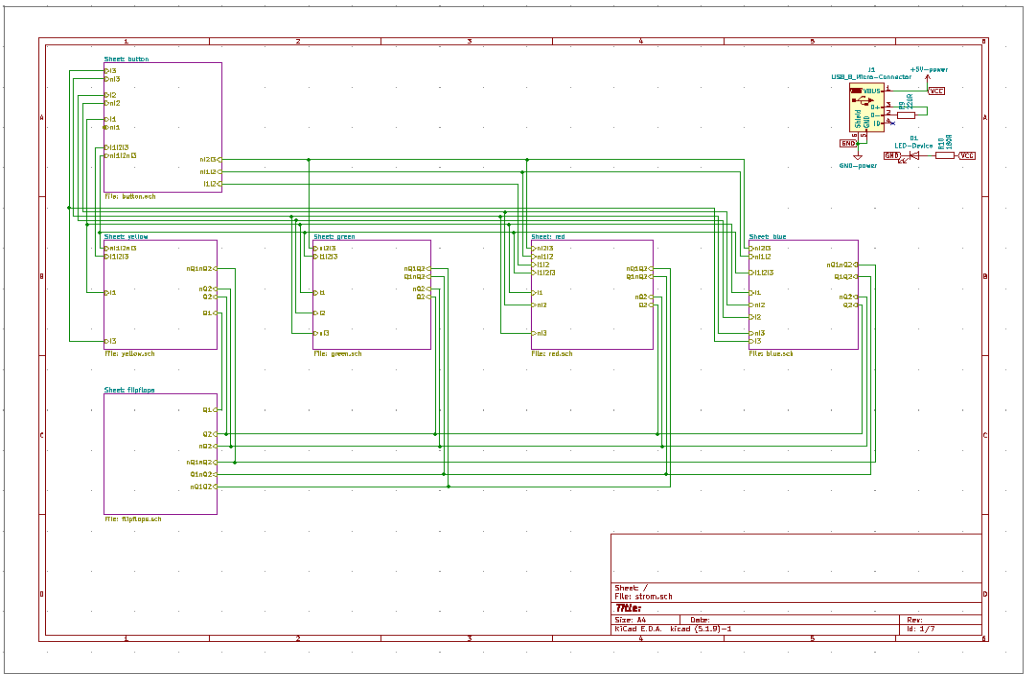

Figure 3.1: Circuit Design in KiCad

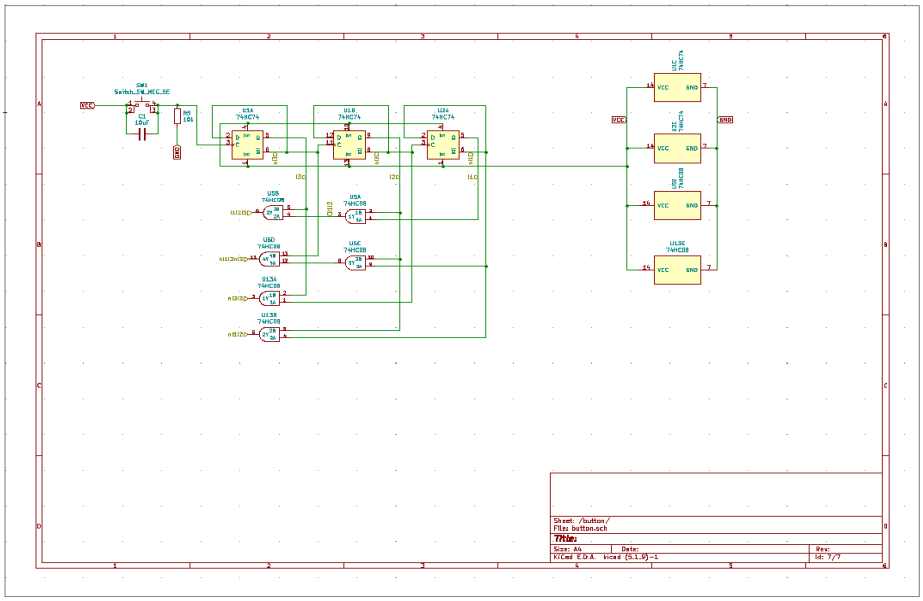

Figure 3.2: Button sheet

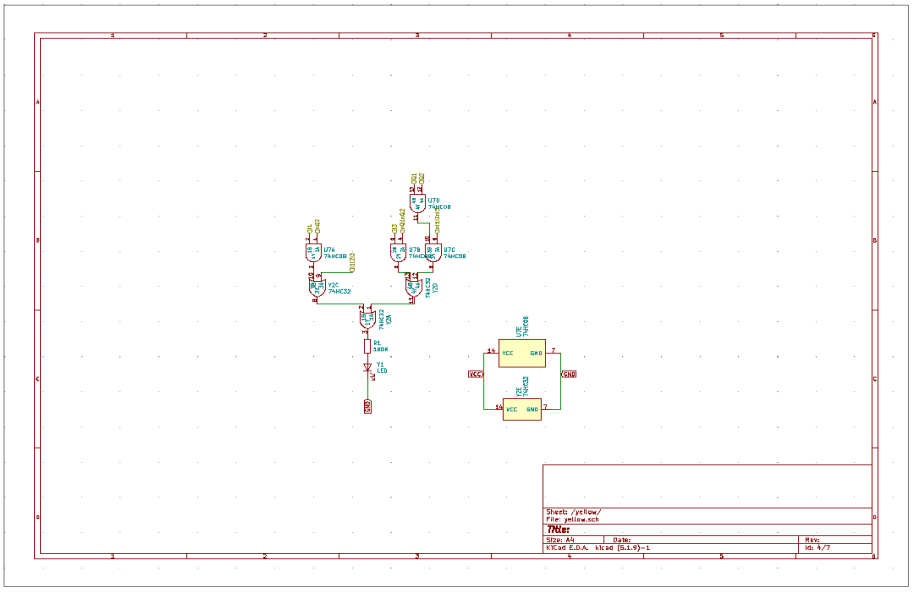

Figure 3.3: Yellow LED sheet

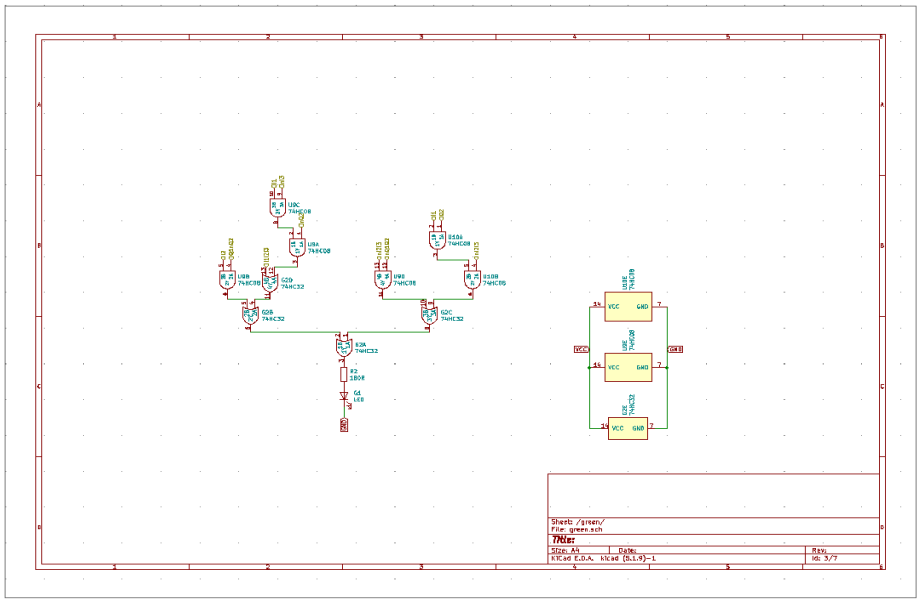

Figure 3.4: Green LED sheet

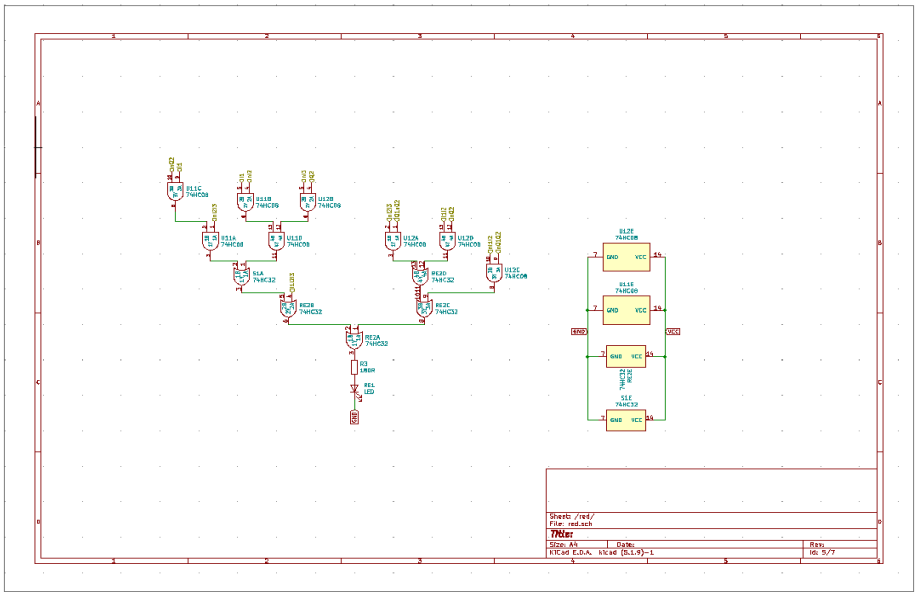

Figure 3.5: Red LED sheet

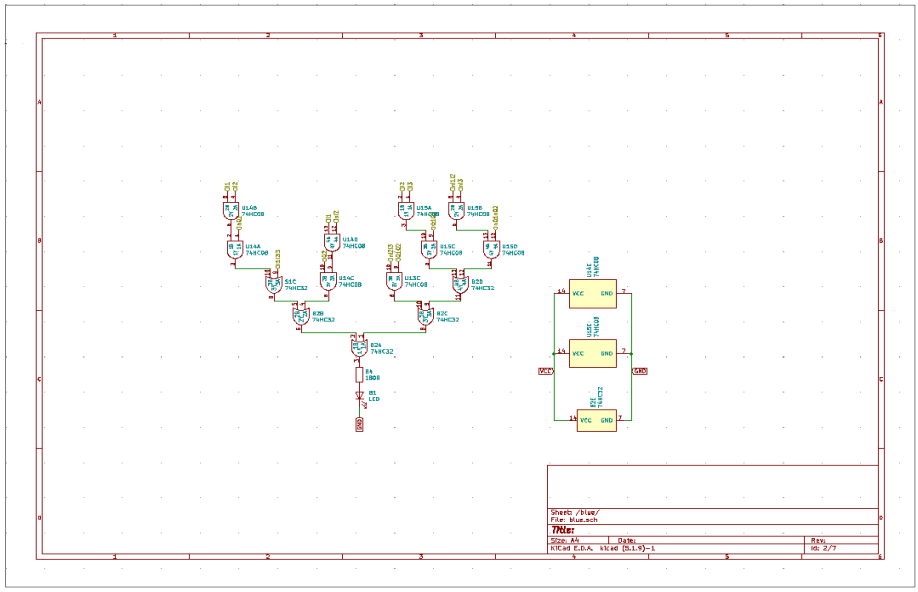

Figure 3.6: Blue LED sheet

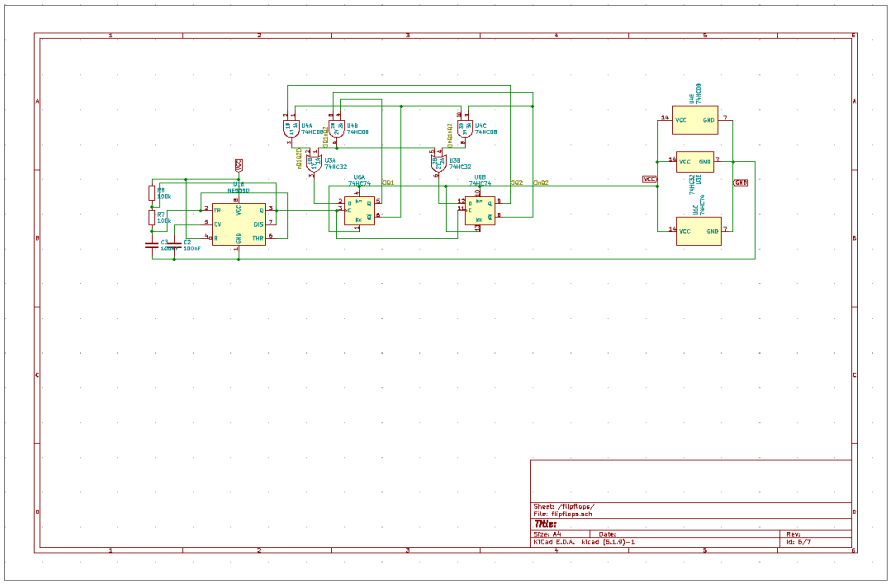

Figure 3.7: State Memorizing D flip flops sheet

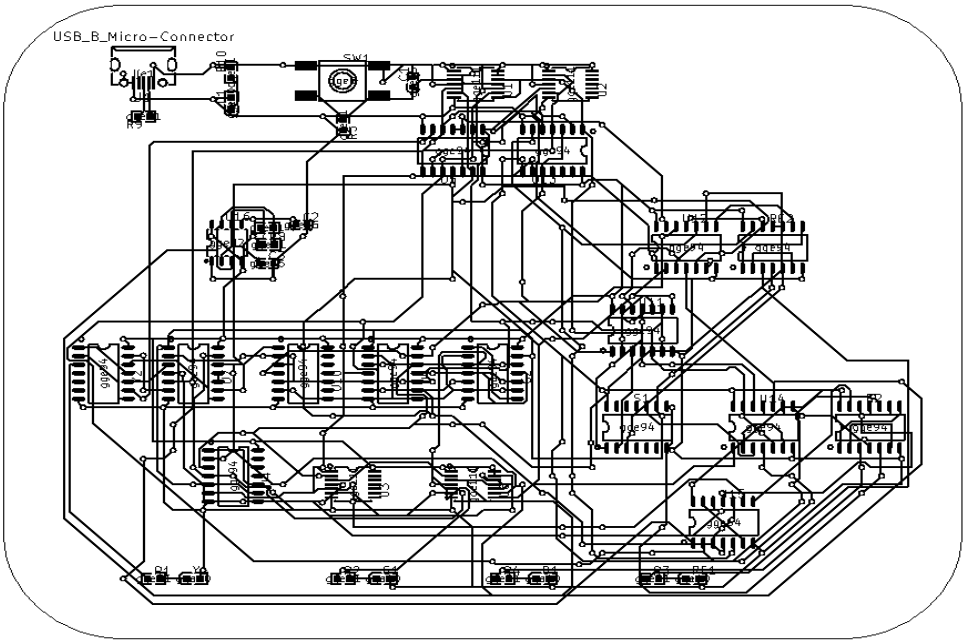

Figure 3.8: PCB Scheme

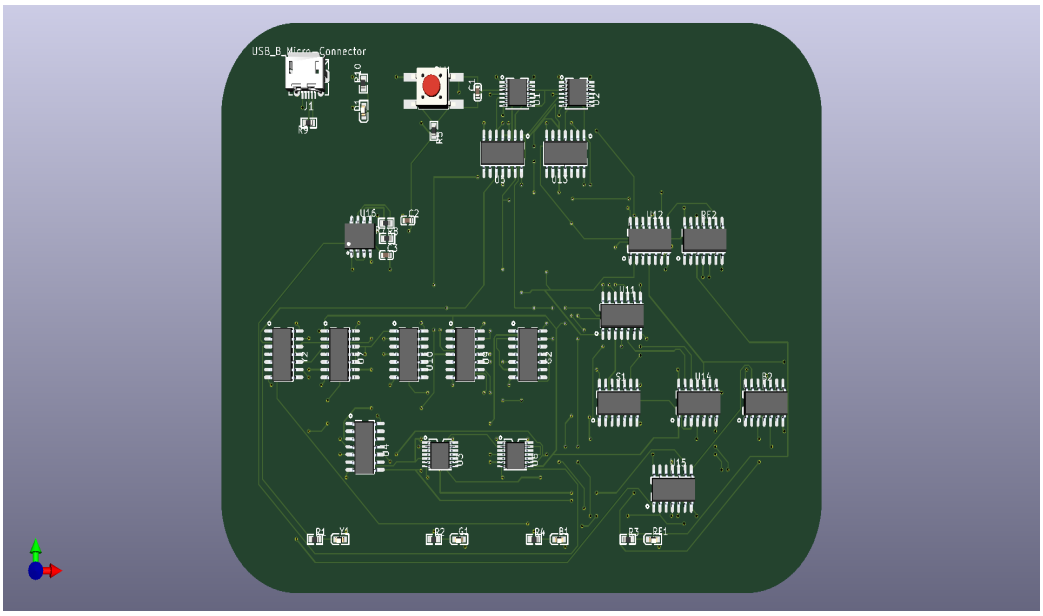

Figure 3.9: 3D Model of the Circuit